QueryAPIGate DOCUMENTATION · PERFORMANCE

Performance on MySQL, v2: what 0.43 changed

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=PERFORMANCE_MYSQL_V2

The case is v1's: one MySQL server holding six databases, one per microservice, and read-only per-user queries that join across them, through one QueryAPIGate connection. v2 measures 0.43 against 0.42, and answers the questions the review of v1 left open.

The short answer. One gunicorn worker, as in the Docker image, 32 concurrent callers:

	V0.42.0	0.43	0.43 + METADATA CACHE
`customer` - one user's row	647/s	743 (+15%)	875 (+35%)
`order_status` - TPC-C Order-Status, 3 tables	573	662 (+16%)	727 (+27%)
`stock_level` - TPC-C Stock-Level, a range and `COUNT(DISTINCT)`	630	751 (+19%)	833 (+32%)
`order_status`, **MySQL 0.5 ms away**	446	**680 (+52%)**	**737 (+65%)**
One caller, `order_status`, MySQL 0.5 ms away: p50	3.71 ms	2.89 ms	2.88 ms

- **0.43 is 15-19% faster with MySQL on the same host, and 52% faster across a network.** A call makes 2 round trips to MySQL instead of 3, and none of them stalls the worker any more (ADR 0027).
- **Threads help again across a network.** On 0.42, more than two threads lost throughput with MySQL 0.5 ms away; on 0.43, four threads serve 730 calls a second, against 0.42's best of 501.
- **Logging is a fifth of a worker.** With gunicorn's access log and QueryAPIGate's request lines both off, one worker served 21% more. Both are settings from 0.43.
- **The laptop's ceiling:** 3,500 `order_status` and 4,400 single-row lookups a second, sustained, from twelve workers on its six performance cores - about 50% more than v1's setting on the same machine.
- **A hyperthread is worth about 0.6 of a core** for the gateway, so one worker per physical core does best.
- **Every one of the 9.5 million requests returned 200.**

Each cell is the median of 5 interleaved rounds, with its min-max range in the tables below. A difference counts as real only when the ranges don't overlap - the method of Performance, v3.

Contents

- Method
- 0.43 against 0.42
- A database across a network
- The review's questions
- Sizing, updated

- Limits
- Run it yourself

Method

As v1: the same laptop (an Intel i7-12700H, on AC power, the `performance` profile and energy preference, a desk fan on it this time), the same TPC-C data at 20 warehouses moved into six databases, the same read-only user, the same queries and load files. One session, 2026-10-26 18:32 to 21:59. Each part on its own cores:

PART	CPUS
QueryAPIGate (gunicorn)	0-7: four performance cores, both threads
MySQL 8.4.13	8-11: two performance cores
The load generator (oha)	12-19: the efficiency cores

The laptop ceiling block moves them; it says how.

LABEL	COMMIT
v0.42.0	`647ab6f`
0.43	`fe603c0`: `main` with everything 0.43 ships but its version number - ADR 0027, the schema-qualified allow-lists, `db._` `target()` cached, the metadata cache defaults, `QUERYAPIGATE_THREADS`
0.43 + cache	the same, with `QUERYAPIGATE_METADATA_CACHE_TTL=5`

Each runs as the Docker image does - gunicorn's `gthread` worker, 8 threads, the access log on - except where a block varies that. The store is SQLite in the home directory, except where a block says PostgreSQL.

Software: Python 3.12.3, gunicorn 26.2.0, mysql-connector-python 9.3.0, MySQL 8.4.13, oha 1.16.0 pinned by digest, go-tpc (snapshot `a9ca481`). Linux 6.8.0.

Steadiness. A `/health` floor before and after every block stayed between 1,983 and 2,311 requests a second. The first rounds of the throughput block ran with some unrelated containers on the machine (about 0.2 of a core, unpinned); they were stopped ten minutes in, before the second query. The builds were interleaved within every query, so each comparison saw the same conditions.

What v2 leaves out. MySQL on its own, the response cache under skewed traffic, the open-loop latency curve and local threads per worker: nothing 0.43 changed affects them, and v1's figures stand. Event-log pruning at steady state needs a run longer than the shortest retention (an hour), and is left to a soak test.

0.43 against 0.42

One worker, 32 callers

QUERY	V0.42.0	0.43	0.43 + CACHE
`customer`	647 (636-693)	743 (726-795)	875 (831-883)
`order_status`	573 (567-604)	662 (644-670)	727 (721-753)
`stock_level`	630 (628-677)	751 (733-795)	833 (829-902)

All the ranges are clear of each other. What 0.43 removed from every call: mysql-connector's ping on each `cursor()` (2.6-3.8% of the CPU in v1's profile, and a stall of the whole worker - see below), `db._target()` re-reading the environment (3.6-3.8%), and #95's two store reads. v1's run had measured v0.42.0 at 643, 558 and 613 for these three: this run agrees with it within 3%.

The gateway still sets the limit: it used 1.2-1.3 cores, and MySQL 0.07-0.52.

One caller at a time

QUERY	V0.42.0	0.43	0.43 + CACHE	THE GATEWAY ADDS (0.43 + CACHE)
`customer`	1.26 ms	1.13 ms	1.07 ms	+1.04 ms
`order_status`	1.47 ms	1.41 ms	1.35 ms	+1.27 ms
`stock_level`	1.71 ms	1.76 ms	1.65 ms	+1.23 ms

The p50 of a single call. "Adds" is against the same query sent straight to MySQL (v1). With nothing else running, one caller gains less: 10% on `customer`, 4% on `order_status`; `stock_level`'s ranges overlap. **A call through the gateway costs 1.0-1.3 ms more than the query alone,** down from v1's 1.1-1.4.

Where the time goes now

py-spy on one worker, `order_status` at 32 callers, as in v1:

	0.43	0.43 + CACHE
Store layer: the key, the query and the connection, from SQLite	23.1%	3.6%
Data query: mysql-connector and the pool	20.0%	22.7%
Logging	14.9%	19.2%
gunicorn, sockets, HTTP	13.8%	16.3%
Flask and Werkzeug	7.0%	9.9%
SQL checks, parameters	5.2%	7.9%
JSON	5.1%	7.3%
History writer thread	5.0%	5.1%
Everything else	5.9%	7.9%

The ping is gone from the profile, and `db._target()` is down from 3.8% of the samples to under 1%. With the cache on, **logging is now the largest share after the query itself** - What logging costs measures turning it off.

A database across a network

netem delays everything MySQL sends by 0.5 ms (about a same-zone cloud round trip), `order_status`:

	V0.42.0	0.43	0.43 + CACHE
One caller: p50	3.71 ms (3.63-3.74)	2.89 ms (2.76-2.90)	2.88 ms (2.84-2.89)
One caller: calls/s	271 (267-278)	349 (347-362)	349 (347-355)
32 callers: calls/s	446 (445-449)	**680 (643-684)**	**737 (713-745)**

- **For one caller, 0.43 saves 0.8 ms a call:** one round trip fewer, about 0.6 ms at this delay, plus the work.
- **For 32 callers, 0.43 serves 52% more** - more than one round trip explains. In 0.42 the round trip it saves was a ping during which mysql-connector held Python's interpreter lock, so every thread waited for it, not just the one calling (v1). 0.43's gateway used 1.31 cores here against 0.42's 0.98: threads that were stalled are working.

Round trips

Counted in MySQL's general log over 100 calls, and measured as the slope of one caller's p50 against delay, calibrated by mysql-connector sending a known one statement (1.27 ms of p50 per ms of delay):

	STATEMENTS PER CALL (LOG)	P50 AT +0 / 0.5 / 1 / 2 MS	SLOPE	ROUND TRIPS
v0.42.0	2: the query, `ROLLBACK`	1.27 / 3.39 / 4.98 / 9.19 ms	3.94	3.1
0.43	2: the query with its time limit as a hint, `ROLLBACK AND CHAIN`	1.12 / 2.60 / 3.65 / 6.47 ms	2.65	**2.1**
0.43 + cache	the same	1.05 / 2.53 / 3.62 / 6.39 ms	2.63	2.1

The log shows the same two statements a call for both; 0.42's third round trip was the ping, which isn't a statement.

Threads per worker, across the network

The block v1 couldn't explain: one worker, `order_status`, 32 callers, MySQL 0.5 ms away:

THREADS	V0.42.0	0.43	0.43 OVER 0.42
1	316 (311-317)	405 (403-408)	+28%
2	**501** (499-507)	648 (640-665)	+29%
4	469 (468-473)	**730** (729-730)	+55%
8	447 (445-449)	685 (680-686)	+53%

- **On 0.42, threads beyond two cost throughput:** each added thread meant more pings, and each ping stopped every thread.
- **On 0.43, threads overlap the network's waiting, as they should:** four serve 46% more than 0.42's best. Eight are a little slower than four on this query, for the reason fewer threads win on the same host (v1): they compete for the interpreter lock.

The store across the network

QueryAPIGate's own store on local SQLite, against PostgreSQL 0.5 ms away (what several instances share); MySQL 0.5 ms away in both; one caller:

STORE	0.43	0.43 + CACHE
SQLite, local	2.86 ms (351/s)	2.82 ms (359/s)
PostgreSQL, 0.5 ms away	**5.70 ms (176/s)**	**2.87 ms (349/s)**

As in v1: **without the cache, a store across the network costs a call 2.8 ms; with it, nothing.**

The review's questions

The review of v1 asked for each of these.

Scale-out on a PostgreSQL store

v1 scaled workers on local SQLite, whose single writer was a suspect at 8 processes. Here the store is PostgreSQL (on the database cores, no delay), as several instances would share it; `order_status`, 64 callers:

WORKERS	0.43	0.43 + CACHE	CALLS/S PER GATEWAY CORE: 0.43 / CACHE	STORE CORES: 0.43 / CACHE
1	625 (620-667)	720 (716-760)	522 / 619	0.24 / 0.07
2	1,008 (1,001-1,008)	1,198 (1,195-1,201)	425 / 520	0.41 / 0.13
4	1,426 (1,420-1,431)	1,745 (1,735-1,753)	305 / 383	0.74 / 0.24
8	1,862 (1,831-1,865)	2,270 (2,233-2,321)	253 / 310	1.15 / 0.37

- **The store wasn't what flattened v1's scale-out.** On PostgreSQL the curve has the same shape: per-core throughput halves from 1 to 8 workers, with the store using at most 1.15 cores. What flattens it is the gateway's cores - next.
- **The cache's lead holds at every size (+15% to +22%),** and it cuts the store's load to a third.

Is a hyperthread worth less than a core?

v1 explained its flattening scale-out by eight workers sharing four cores' hyperthreads. Tested directly: 4 workers, cache on, `order_status`, 64 callers, on three sets of CPUs (CPUs come in sibling pairs, 0-1, 2-3, ...):

4 WORKERS ON	CPUS	CALLS/S	P99
one thread of each of 4 cores	`0,2,4,6`	**2,250** (2,243-2,256)	59 ms
both threads of 2 cores	`0-3`	1,807 (1,777-1,828)	72 ms
both threads of 4 cores	`0-7`	1,756 (1,749-1,767)	55 ms

- **Two cores with both hyperthreads did 80% of four cores' work:** a core's second hyperthread is worth about 0.6 of a core here. So v1's explanation stands, with a number on it.
- **Four workers on eight CPUs were slower than on four.** With twice the CPUs they need, the scheduler sometimes puts two busy workers on one core's two hyperthreads. Pin one worker per physical core where you can.

What logging costs

0.43 + cache, `order_status`, 32 callers:

LOGGING	CALLS/S	VS THE IMAGE
gunicorn's access log and QueryAPIGate's request lines (the image's default)	722 (717-754)	
request lines only (`QUERYAPIGATE_ACCESS_LOG=off`)	765 (763-766)	+6%
neither (and `QUERYAPIGATE_LOG_REQUESTS=off`)	**874** (873-903)	**+21%**

Turning both off is worth a fifth of a worker - where a proxy or load balancer logs requests already, they duplicate it. Errors, warnings, slow queries and agent writes are still logged with both off.

The light-load tail

v1 saw a p99.9 of 10-30 ms even at a quarter of capacity, and asked whether it lines up with the history writer's flush, once a second. Every call's latency, at 150 calls a second for 60 s, measured from when each call was due:

HISTORY FLUSH	P50	P99	P99.9	CALLS OVER 10 MS	IN HOW MANY OF THE 60 SECONDS
every 1 s (the default)	2.61 ms	3.37 ms	15.9-26.6 ms	13-29	4-9
every 5 s	2.62 ms	3.36 ms	21.3-26.5 ms	16-25	5-7

- **p99 is 3.4 ms at light load;** the p99.9 is a handful of slow moments, not a slow tail.
- **The slow calls come in bursts:** four or five calls in a row (one every 6.7 ms at this rate) each take 25-35 ms, every five seconds or so - so something stops the worker for about 30 ms at a time.
- **It isn't the history writer.** Flushing every 5 s instead of every 1 s changed neither how often nor how long.
- **It isn't the garbage collector either, mostly.** Run again with every collection the gateway's worker and the load client made logged against the clock: in three runs, one burst each lined up with a 10 ms full collection in the worker; the other two to five bursts per run had no collection near them, in the worker or the client. None of QueryAPIGate's own timers runs every five seconds (the instance heartbeat is 30 s, the retention sweep 10 minutes).

So the light-load p99.9 is not yet explained. What's ruled out: the history writer, the garbage collector for all but a third of it, and the server's timers. Left to look at: gunicorn's request loop (its main thread hands each keep-alive connection back to the worker threads) and MySQL's own background work. It costs a few calls in a thousand about 30 ms each; p99 is unaffected.

Round trips to a PostgreSQL store

v1's store across the network cost 2.54 ms a call at 0.5 ms, which looked like 4 round trips for 3 reads. One caller, 0.43 without the cache, delay on the store only:

STORE DELAY	0	0.5 MS	1 MS	2 MS
p50	1.76 ms	4.21 ms	6.00 ms	10.08 ms

The slope is 4.10 ms per ms of delay: 3.2 round trips at mysql-connector's calibration above, 3.6 at the 1.15 v3 measured for PostgreSQL. **The store makes 3 reads, each one statement in autocommit** - the key, the saved query, the connection - **and so 3 round trips;** the first half-millisecond of delay costs more than the slope (2.45 ms, not 2.05), which is what made v1's single point look like four. With the metadata cache, none of the three is made.

The laptop's ceiling

What this laptop can serve, against v1's setting on the same machine. Both with the cache on, 64 callers:

	V1'S SETTING	THE CEILING
Gateway	8 workers x 8 threads on CPUs 0-7 (four performance cores)	12 workers x 2 threads on 0-11 (all six)
Logging	both on	both off
MySQL	8-11 (two performance cores)	12-17 (six efficiency cores)
Load generator	12-19	18-19

QUERY	V1'S SETTING	THE CEILING	
`customer`	2,927 (2,876-4,029)	**4,382** (4,372-6,565)	+50%
`order_status`	2,376 (2,318-3,060)	**3,507** (3,490-3,935)	+48%
`stock_level`	2,532 (2,507-3,521)	**3,514** (3,382-3,522)	+39%

- **Sustained, the laptop serves about 3,500 multi-database joins and 4,400 single-row lookups a second.** The gateway used 11 of its 12 hardware threads.
- **`stock_level` is now MySQL-bound:** MySQL used 5.4 of its six efficiency cores.
- **The highest value in each range is the first round, at boost clocks.** After a lighter block, the performance cores ran at 3.3-3.7 GHz for the first few minutes, then settled at 2.1-2.6 GHz under the laptop's sustained power limit; the four cells measured at the higher clocks came out 29-50% above the rest. The medians are the sustained figures. A server without a laptop's power limit would hold the higher clocks.

Sizing, updated

v1's rule, with 0.43's figures:

- **CPU:** plan on about 500 `order_status`-class calls a second per worker, or 545 with the metadata cache, and 650 with both request logs off as well. That's 75% of this run's capacity.
- **Waiting:** a worker serves at most threads ÷ (query time + round trips × network round-trip time) calls a second, with **2 round trips to MySQL** (3 before 0.43), plus 3 to a PostgreSQL store, none with the cache.
- **Threads:** with MySQL on the same host, 1-2 for fast lookups (v1); with MySQL across a network, 4 - from 0.43, threads overlap that waiting.
- **Workers needed** = the larger of the two limits divided into your calls a second, plus one spare; one worker per physical core.

Tune a deployment for throughput puts these together with PostgreSQL's.

Limits

- **One machine.** Several hosts are still unmeasured: the scale-out is workers sharing one laptop's cores.
- **A laptop's power limit** shapes every multi-worker number here (see the ceiling). A server would sustain more.
- **Every query is cheap,** as in v1: the data fits MySQL's buffer pool.
- **The network is simulated:** one-way delay, no jitter or loss.
- **Event-log pruning at steady state** wasn't reached; that needs a soak of more than an hour.

- **The light-load p99.9** - bursts of about 30 ms every few seconds - is measured, not explained (see The light-load tail).
- **p99.99** wasn't measured.

Run it yourself

As v1, with session `C`:

```shell
pip install -e ".[server,mysql,postgres]" py-spy
python benchmarks/tpcc_mysql.py --out /tmp/qag-tpcc --go-tpc ./go-tpc --refs v0.42.0,origin/main \
    --m2-queries customer,order_status,stock_level setup C teardown   # about 3.5 hours
python benchmarks/tpcc_mysql_report.py /tmp/qag-tpcc/results.jsonl
```

Session `C` runs the comparison (`m2`, `m3`, `m8` - `m11`) and the v2 blocks: `n1` threads across the network, `n2` scale-out on a PostgreSQL store, `n3` hyperthreads, `n4` logging, `n5` the light-load trace, `n6` the store's round trips, `n9` the ceiling.

The 2026-10-26 run is in `benchmarks/results/tpcc-mysql-2026-10-26/`.

READ NEXT
Performance, v3: an OLTP benchmark on PostgreSQL

QUESTIONS OR PROBLEMS
Discord · GitHub issues

LATEST VERSION OF THIS PAGE
anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=PERFORMANCE_MYSQL_V2

FOLLOW QUERYAPIGATE GitHub X Discord Bluesky Mastodon YouTube Reddit
@queryapigate on X, Bluesky, Mastodon and YouTube · r/queryapigate on Reddit · github.com/AnanthaRajuC/QueryAPIGate

QueryAPIGate · one governed gate between your databases and everything that asks them for data · self-hosted, Apache-2.0 · © 2026 QueryAPIGate